

HANDBOOK · V0.2.0

# Getting things built with redgear

Point it at a repository, give it a plan you have read, and let it work. This is everything you need for your first run and the ones after it.

CONTENTS

What you get

---

Install

---

Reading the output

---

Configuration

---

Cost and stopping

---

Error messages

---

What it won't do

---

Before you start

---

Your first run

---

Writing a plan that works

---

Your git repository

---

When a task fails

---

Command reference

---

---

## What you get

redgear drives a coding agent through a plan, one task at a time, and checks every result itself before moving on.

You write down what you want. redgear turns that into a task graph, you read and approve it, and then it works through the graph: composing a prompt, handing it to your coding agent CLI, and then — after that process has exited — **running your tests, re-hashing the files the agent was told not to touch, and recomputing the real `git diff`** to see what actually changed.

If the evidence disagrees with the agent's report, the task does not advance. If it agrees, redgear commits that task and moves to the next one. You get a repository where every commit is one verified piece of work, carrying the proof that verified it.

### THE PART PEOPLE MISS

redgear never reads your API key and never calls a model. It runs your agent CLI as a subprocess, authenticated however you already set it up. redgear itself costs nothing to run — all spend is on your existing agent subscription.

---

## Before you start

| YOU NEED             | WHY                                                                                           |
|----------------------|-----------------------------------------------------------------------------------------------|
| Python 3.12+         | redgear runs on it. Your project can be anything, but the built-in harness is Python.         |
| git                  | Verification is built on real diffs. A project that isn't a git repository can't be verified. |
| A clean working tree | Uncommitted changes make the diff audit meaningless, so runs refuse to start on one.          |
| Claude Code          | The reference agent CLI, already signed in.                                                   |

### IF YOU INSTALLED CLAUDE CODE AS THE DESKTOP APP ON WINDOWS

The `claude` binary is deliberately **not on your PATH**, so redgear can't find it by name. Point it at the binary directly the first time:

```
$ redgear run --executable "C:\Users\you\AppData\Local\Packages\...\claude.exe"
```

Or set it once in `.redgear/config.json` under `runner.executable` and never think about it again. `redgear doctor` tells you which one it resolved.

---

## Install

```
$ pipx install redgear
```

Then check your environment before doing anything else:

```
$ redgear doctor
```

```
+-----+-----+
| check          | result          |
+-----+-----+
| redgear        | 0.2.0           |
| agent CLI (claude) | 2.1.237 (Claude Code) |
| harness: lint   | available       |
| harness: tests  | available       |
| harness: coverage | available       |
| harness: coverage source | src exists    |
| git HEAD       | 3df2ceae1f9b    |
| git tree       | clean          |
+-----+-----+
```

Every row should be green before your first run. The two that catch people out are the agent CLI (see the warning above) and the coverage source — that one has to name a directory that actually exists in your project, or every implementation task will fail for a reason that has nothing to do with your code.

### UPGRADING FROM 0.1.0? DO IT.

0.1.0 cannot complete a plan with more than one task. Nothing committed between tasks, so the second task saw the first one's work as an unexpected change and failed every attempt. Fixed in 0.2.0.

---

## Your first run

Seven steps. The fourth one is the one that matters, and it is the only one redgear cannot do for you.

---

**01**

## Set up the repository

```
$ cd my-project
redgear init
```

This creates `.redgear/` — the plan, the event log, and every proof live here. It is committed to your repo like any other file, because it is the record of what happened.

It also writes a starter `.redgear/config.json` with your harness commands filled in and a guess at where your code lives. **Open it and check that guess.**

---

**02**

## Write down what you want

A plain markdown file. Requirements, each with something you could actually check, plus an explicit list of what is *not* in scope. Section 6 covers how to write this well — it has more effect on your results than anything else you do.

```
# Ledger

## FR-1 Postings must balance
A posting with debits != credits is rejected with UnbalancedPosting.
- post([+100, -100]) succeeds
- post([+100, -50]) raises UnbalancedPosting

## Out of scope
- Multi-currency
- Any HTTP interface
```

---

**03**

## Generate the plan

```
$ redgear plan --from docs/requirements.md
```

This dispatches one **read-only** agent turn — it can read your repo but cannot edit a single file — and turns your document into a task graph: which pieces of work, in what order, with what acceptance criteria.

The plan lands in `.redgear/task_graph.json` in state `draft`.

## Read the plan. Actually read it.

```
$ redgear status
```

```
+-----+-----+-----+-----+-----+
| task   | type           | state  | att  | blocked by |
+-----+-----+-----+-----+-----+
| T-0001 | test_authoring | ready  | 0/3  | -          |
| T-0002 | implementation | blocked | 0/3  | T-0001     |
| T-0003 | test_authoring | blocked | 0/3  | T-0002     |
+-----+-----+-----+-----+-----+
```

Open `.redgear/task_graph.json` and check four things:

- **Do the acceptance criteria describe what you actually want?** These become the tests. Everything downstream is measured against them.
- **Is the scope narrow?** A task writable across your whole source tree is a planning failure. Re-plan rather than accept it.
- **Is `out_of_scope` filled in?** This is what stops an agent helpfully building things you never asked for.
- **Does the order make sense?** Each implementation task should follow the test task it inherits from.

### THIS IS THE ONLY STEP WITH NO SAFETY NET

Every task afterwards is checked against tests — but the plan *defines* those tests. A wrong plan produces software that is confidently verified and still wrong, and no amount of checking downstream catches it. There is deliberately no flag that skips this step.

## Approve it

```
$ redgear approve --by "your name"
```

This records who approved which version of the requirements. Edit the requirements afterwards and the approval is invalidated — you'll be asked to approve again, because it is no longer the plan you read.

06

## See the prompts before you pay for them

```
$ redgear run --dry-run
```

Composes and prints every prompt the loop would send, and dispatches **nothing**. It costs nothing, writes nothing, and is the fastest way to catch a badly scoped task. Use it constantly.

Read the SCOPE and ACCEPTANCE sections of the first prompt. If they look wrong here, they will produce wrong work.

07

## Run it

```
$ redgear run
```

```
redgear run
  stop with: redgear stop (or create .redgear/STOP)

agent CLI: claude (2.1.237)
spend cap: $1.00 per dispatch

complete: 6 iteration(s), 6 verified, 0 escalated
```

That is the whole console for a clean run: the brake, the agent, and one summary line. Everything else is in the audit trail, because the point isn't a chatty console — it's a record you can check afterwards.

# Reading the output

## Where everything lives

| PATH                                                                   | WHAT IT HOLDS                                                     |
|------------------------------------------------------------------------|-------------------------------------------------------------------|
| <code>.redgear/task_graph.json</code>                                  | Current state of every task. A projection — the log is the truth. |
| <code>.redgear/events.jsonl</code>                                     | Append-only record of everything that happened, in order.         |
| <code>.redgear/runs/&lt;run&gt;/iterations/&lt;n&gt;/prompt.txt</code> | The exact prompt that was sent. Every one is kept.                |
| <code>.../proof/verdict.json</code>                                    | Every gate, its result, and why.                                  |

| PATH                              | WHAT IT HOLDS                                         |
|-----------------------------------|-------------------------------------------------------|
| <code>.../proof/diff.patch</code> | What actually changed, captured at verification time. |

## The commands that read it

```
redgear status      # where every task stands
redgear log --tail 20 # what just happened, secrets redacted
redgear rebuild     # verify the log and the state still agree
```

`rebuild` replays the log from the beginning and compares the result against what's on disk. If they disagree it says so and changes nothing — it never repairs. An audit trail that quietly fixes itself isn't one.

## What a commit looks like

```
T-0002: implement posting validation

Verified by redgear. Attempt 2 of 3.

task: T-0002
type: implementation
run: run_20260822T004524033401
proof: .redgear/runs/run_.../iterations/0002/proof
gates: scope_check frozen_hash_check lint tests_pass criteria_coverage coverage_delta
spec: sha256:97ee71867c3867b80290dfd89c89d4c1dcb...
```

Your git identity, no signature, no co-author. The trailers let anyone reading `git log` in a year find the proof.

---

## Writing a plan that works

Plan quality determines result quality more than any setting. These are the things that consistently matter.

### Write criteria you could check by hand

"Handles errors gracefully" cannot become a test. "Raises `UnbalancedPosting` when debits and credits differ" can. Every criterion becomes a test node, and a vague criterion becomes a vague test that passes when it shouldn't.



## Say what is out of scope

This is the field that stops an agent building a caching layer you never asked for. Be specific and slightly over-inclusive — naming something you were never going to build costs nothing.

## Prefer many small tasks to few large ones

Small tasks have narrow scope, which means the freeze protects more, failures are easier to diagnose, and a retry re-does less work. If a prompt hits redgear's 8,000-character ceiling, that's a signal the task is too big — split it.

## Understand the pairing

| TYPE           | CAN WRITE             | FROZEN          | PASSES WHEN                                           |
|----------------|-----------------------|-----------------|-------------------------------------------------------|
| test_authoring | tests                 | your source     | The tests exist and <b>fail</b> for the stated reason |
| implementation | your source           | tests           | Those same tests <b>pass</b>                          |
| scaffold       | config, CI, packaging | usually nothing | Smoke checks succeed                                  |

The test task writes the tests and cannot touch your source. The implementation task makes them pass and **cannot edit a single character** of those tests — it inherits them. That is what stops an agent making a failing test pass by changing the test.

REVIEWING A PLAN QUICKLY

Run `redgear run --dry-run` and read the first two prompts. If the scope and criteria look right there, the plan is probably fine. It costs nothing.

---

## Configuration

`.redgear/config.json`, written by `init` and yours to edit.

```

{
  "runner": {
    "executable": "claude",
    "bare": false
  },
  "commands": {
    "lint_cmd": ["python", "-m", "ruff", "check", "--output-format=json", "--no-cache", "
    "test_cmd": ["python", "-m", "pytest"],
    "coverage_cmd": ["python", "-m", "coverage"],
    "coverage_source": ["src"]
  },
  "gates": {
    "coverage_floor": 0.85,
    "timeout_s": 900
  }
}

```

| SETTING                        | WHAT IT DOES                                                                                 |
|--------------------------------|----------------------------------------------------------------------------------------------|
| <code>runner.executable</code> | Path to your agent CLI. Set this if it isn't on your PATH.                                   |
| <code>commands.*</code>        | The commands redgear runs to check work. Point these at whatever your project actually uses. |
| <code>coverage_source</code>   | <b>The directory your code lives in.</b> Must exist, or coverage measures nothing.           |
| <code>coverage_floor</code>    | Minimum coverage of the lines a task changed. Not your whole project — just what it touched. |
| <code>timeout_s</code>         | How long any single harness command may run.                                                 |

#### GET `COVERAGE_SOURCE` RIGHT FIRST

If it names a directory that doesn't exist, coverage collects nothing and every implementation task fails on the coverage gate — after paying for the dispatches that got there. `redgear doctor` checks this for you, which is why it is worth running before your first task.

## What redgear does to your git repository

Four behaviours. Two of them will surprise you, so they're worth reading before your first run rather than discovering afterwards.

## It commits each verified task

Locally, with your git identity, one commit per verified task carrying its proof in the message. This is also what makes verification work — each task is measured against the commit the previous one produced.

## It bypasses your pre-commit hooks

Commits use `--no-verify`. A hook that reformats would change your code *after* redgear computed the proof, so the commit would contain code that was never checked. redgear has already run your own lint and test commands as part of verification.

## It restores the tree after a failed attempt

So the retry starts clean instead of on top of half-finished work. It only ever undoes the turn that just failed: the tree is confirmed clean before every task, `.redgear/` is never touched, and ignored files — your virtualenv, your build output — are never removed.

## It leaves a failed task's work alone

When a task gives up, nothing is committed and nothing is reverted. That state is the evidence you need to diagnose it. `redgear status` tells you what's sitting there and how to clear it when you're done.

### IT NEVER TOUCHES SHARED HISTORY

No pushing, no rebasing, no resetting, no rewriting. Your branches and your remote are yours.

## Undoing a task

Restore that task's files from the commit before it:

```
$ git checkout <sha>^ -- src/the/paths/it/wrote
```

A plain `git revert` will conflict on the event log. That's deliberate — the log is append-only, and reverting would delete history that later entries were written on top of. Undo the work; leave the record.

---

## Cost and stopping

### Stopping a run

```
$ redgear stop
```

The loop finishes what it's doing, releases its lease, and exits cleanly. It checks before every task, so a stop never leaves things half-done. Ctrl-C works too — it kills the agent process, records that it was interrupted, and releases its locks.

### What a run costs

redgear itself costs nothing. The spend is your agent CLI's, and in practice a dispatch has measured between **\$0.07 and \$0.35**, so a small multi-task run lands around a dollar. Your mileage depends on task size.

### The caps

| CAP                  | DEFAULT | WHAT IT PROTECTS AGAINST                              |
|----------------------|---------|-------------------------------------------------------|
| Spend per dispatch   | \$1.00  | One runaway turn                                      |
| Iterations           | 50      | A loop that never finishes                            |
| Wall clock           | 2 hours | A run you forgot about                                |
| Consecutive failures | 5       | A broken environment burning money on doomed attempts |
| Time per dispatch    | 15 min  | A hung agent process                                  |
| Attempts per task    | 3       | Retrying something that isn't going to work           |

Caps are checked between tasks, never mid-task, so hitting one never leaves work in a strange state. Override iterations for a single run with

```
--max-iterations
```

---

## When a task fails

A failed task goes back in the queue, and the next prompt for it carries the actual failure — the file, the line, the assertion — so the retry is corrective rather than a blind repeat. You'll see the attempt count climb:

```
| T-0007 | implementation | rejected | 1/3 | - |
```

After three attempts, or if the agent reports it genuinely cannot proceed, the task escalates and the run pauses for you:

```
| T-0007 | implementation | escalated | 3/3 | - |
```

**escalated: T-0007 (needs a human)**

T-0007: attempts\_exhausted

working tree carries 2 uncommitted path(s) from the failed attempt  
inspect the proof, then discard with:

```
git restore --staged --worktree . && git clean -fd
```

## Diagnosing it

1. Open that attempt's `proof/verdict.json`. It names every gate and which one failed first.
2. Open `proof/diff.patch` to see exactly what the agent did.
3. Open the matching `prompt.txt` to see what it was told.

Most escalations are one of three things: a criterion that was ambiguous, a scope that was too narrow for the work, or a harness command that isn't right for your project. All three are fixed in the plan or the config, not by retrying.

### "BLOCKED" IS NOT A FAILURE

An agent that reports it can't proceed honestly costs you nothing — no attempt is consumed. That's deliberate: if honesty were expensive, a stuck agent would be better off guessing, and a confident guess is the thing redgear exists to catch. Treat `blocked` as useful information about your plan.

## Which gate failed, and what it means

| GATE                           | MEANS                                                                                                        |
|--------------------------------|--------------------------------------------------------------------------------------------------------------|
| <code>scope_check</code>       | It changed a file outside its scope, or its report of what it changed didn't match reality.                  |
| <code>frozen_hash_check</code> | It modified, added to, or deleted from a frozen path.                                                        |
| <code>lint</code>              | Your linter flagged something in the files it wrote.                                                         |
| <code>tests_pass</code>        | Tests failed. For a test-authoring task, it means the opposite: the tests didn't fail when they should have. |
| <code>criteria_coverage</code> | A criterion's test didn't run, or didn't pass.                                                               |
| <code>coverage_delta</code>    | The lines it changed aren't covered enough.                                                                  |

---

## Error messages

Every error carries a code. These are the ones you'll actually meet.

| CODE                               | WHAT HAPPENED                                                      | WHAT TO DO                                                       |
|------------------------------------|--------------------------------------------------------------------|------------------------------------------------------------------|
| <code>E_DIRTY_TREE</code>          | Uncommitted changes                                                | Commit or stash them. Verification needs a clean baseline.       |
| <code>E_PLAN_UNREVIEWED</code>     | The plan is still a draft                                          | Read it, then <code>redgear approve</code> .                     |
| <code>E_HARNESS_ERROR</code>       | A configured command is wrong or its coverage source doesn't exist | Fix <code>config.json</code> ; run <code>redgear doctor</code> . |
| <code>E_NOT_A_REPO</code>          | Not a git repository root                                          | Run from the top of your repo.                                   |
| <code>E_RUN_LOCKED</code>          | Another run holds the lock                                         | Wait, or stop the other run.                                     |
| <code>E_SPEC_DRIFT</code>          | Requirements changed after planning                                | Re-plan and re-approve.                                          |
| <code>E_COMMIT_FAILED</code>       | git refused the commit                                             | Usually a missing <code>user.email</code> . Your work is safe.   |
| <code>E_PROJECTION_DIVERGED</code> | State and log disagree                                             | Stop. Something edited state directly. Nothing is auto-repaired. |

Exit codes

| CODE | MEANING                                              |
|------|------------------------------------------------------|
| 0    | Everything verified, or you stopped it cleanly       |
| 2    | A task escalated — needs you                         |
| 3    | A budget cap was reached                             |
| 4    | The agent CLI couldn't be run or understood          |
| 5    | Internal problem — the plan or state is inconsistent |

Command reference

| COMMAND                                       | DOES                                               |
|-----------------------------------------------|----------------------------------------------------|
| <code>redgear init</code>                     | Set up <code>.redgear/</code> and a starter config |
| <code>redgear plan --from &lt;file&gt;</code> | Turn requirements into a draft task graph          |

| COMMAND                                            | DOES                                                  |
|----------------------------------------------------|-------------------------------------------------------|
| <code>redgear approve --by &lt;name&gt;</code>     | Approve the draft so it can run                       |
| <code>redgear run</code>                           | Work through the plan                                 |
| <code>redgear run --dry-run</code>                 | Print every prompt, send nothing, cost nothing        |
| <code>redgear run --max-iterations N</code>        | Cap this run's tasks                                  |
| <code>redgear run --executable &lt;path&gt;</code> | Point at your agent CLI                               |
| <code>redgear status</code>                        | Where every task stands                               |
| <code>redgear stop</code>                          | Stop cleanly at the next boundary                     |
| <code>redgear log --tail N</code>                  | Recent events, secrets redacted                       |
| <code>redgear verify &lt;task&gt;</code>           | Run the checks manually — useful for debugging config |
| <code>redgear rebuild</code>                       | Confirm the log and the state still agree             |
| <code>redgear doctor</code>                        | Check your environment                                |
| <code>redgear ui</code>                            | Serve the read-only API                               |

## What it won't do

Stated plainly, so you find out here rather than mid-run.

| LIMIT                                | DETAIL                                                                                                          |
|--------------------------------------|-----------------------------------------------------------------------------------------------------------------|
| <b>Python projects only</b>          | The built-in harness is pytest, ruff and coverage. Other languages need harness support that doesn't exist yet. |
| <b>One task at a time</b>            | No parallelism. Tasks run in dependency order, sequentially.                                                    |
| <b>Local only</b>                    | Your machine, your repo, your agent CLI. Nothing hosted, nothing remote.                                        |
| <b>Plans are never auto-approved</b> | A human reads every plan before it runs. There is no flag.                                                      |
| <b>No pushing or PRs</b>             | Local commits only. Sharing your work stays your decision.                                                      |
| <b>No browser dashboard yet</b>      | The read-only API exists; the interface on top of it doesn't.                                                   |

#### HONEST ABOUT THE LOOP ITSELF

redgear has driven a real multi-task project to completion unattended, verified and committed, with a replayable audit trail. It has **not** yet been run against a long or complex plan, and the corrective-retry path has been exercised against a simulated agent rather than a live one. Start with a small plan and read the proofs.

---

redgear v0.2.0 · MIT · Python 3.12+

`pipx install redgear · redgear doctor · redgear run --dry-run`